Analysis of Algorithm (CSN13400)
ASSIGNMENT-2

Name : Aayansh Niranjan Date :26/07/2026
Reg_No.: 20253004

Q1. Write a C program that reads an input file containing an iterative algorithm (only for loop-based, no recursion) and computes its time complexity by counting the total number of iterations executed.

A1. CODE:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <math.h>

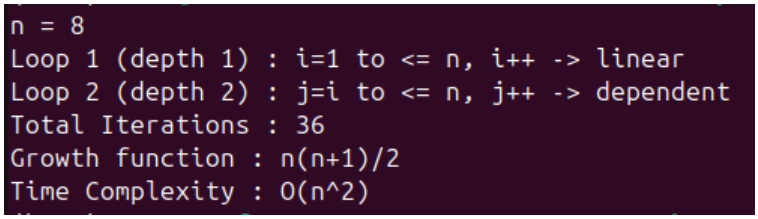
int growthfunc=0;
//1-n
//2-log n
//3-n log n
//4-n^2
//5-n(n+1)/2
int type_val=0;

int main(){
  FILE *file=fopen("Algo.txt","r");
  if(!file){printf("Not available");return 0;}
  char buffer[1000];
  int n;
  fgets(buffer,1000,file);
  sscanf(buffer,"n = %d",&n);
  int depth=0;
  int cnt=0;
  printf("n = %d\n",n);
  int iterations;
  while(fgets(buffer,1000,file)){
    if(strstr(buffer,"{")){depth++;}
    if(strstr(buffer,"}")){depth--;}
    if(strstr(buffer,"for")){
      cnt++;
      char variable;
      char start[100];
      char end[100];
      char update[100];
      char type[100];
      sscanf(buffer,"for (%c = %[^;]; %*c%*c %[^;]; %[^)])",&variable,start,end,update);
      if(strchr(update,'*')||strchr(update,'/')){
          strcpy(type,"logarithmic");
          type_val=1;
          iterations+=ceil(log(n)/log(2));
      }
```

```
    if(strchr(start,'1')){strcpy(type,"linear");iterations+=n;type_val=2;}
    else{strcpy(type,"dependent");iterations=(n*(n+1))/2;type_val=3;}

    if(type_val==1){growthfunc+=2;}
    else if(type_val==2){
      if(growthfunc==0){growthfunc=1;}
      if(growthfunc==1){growthfunc=4;}
      if(growthfunc==2){growthfunc=3;}
    }
    else{
      growthfunc=5;
    }
    printf("Loop %d (depth %d) : %c=%s to %s, %s ->
%s\n",cnt,depth,variable,start,end,update,type);
   }
  }
  printf("Total Iterations : %d\n",iterations);
  switch(growthfunc){
   case 1:
    printf("Growth function : n\nTime Complexity : O(n)\n");
    break;
   case 2:
    printf("Growth function : log2(n)\nTime Complexity : O(log n)\n");
    break;
   case 3:
    printf("Growth function : n log2(n)\nTime Complexity : O(n log n)\n");
    break;
   case 4:
    printf("Growth function : n^2\nTime Complexity : O(n^2)\n");
    break;
   case 5:
    printf("Growth function : n(n+1)/2\nTime Complexity : O(n^2)\n");
  }
  fclose(file);
}
```

OUTPUT:



Q2. Write a C program to implement Linear Search on an array.
A2. CODE:

```
#include <stdio.h>

int main() {
    int arr[100], n, key, i;
    printf("Enter the number of elements: ");
    scanf("%d",&n);
    printf("Enter the elements:\n");
    for(i=0;i<n;i++){
        scanf("%d", &arr[i]);
    }
    printf("Enter the element to search: ");
    scanf("%d",&key);
    for(i=0;i<n;i++){
        if(arr[i]==key){
            printf("Element found at position %d\n",i+1);
            return 0;
        }
    }
    printf("Element not found.\n");
    return 0;
}
```

OUTPUT:

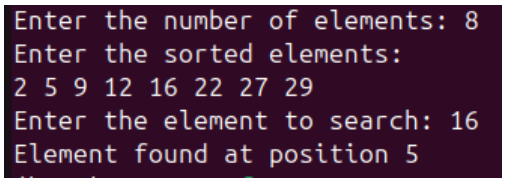
```
Enter the number of elements: 5
Enter the elements:
4 9 2 6 5
Enter the element to search: 6
Element found at position 4
```

Q3. Write a C program to implement Binary Search on a sorted array.
A3. CODE:

```
#include <stdio.h>

int main() {
    int arr[100],n,key;
    printf("Enter the number of elements: ");
    scanf("%d",&n);
    printf("Enter the sorted elements:\n");
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    printf("Enter the element to search: ");
    scanf("%d", &key);
    int low=0;
    int high=n-1;
    while(low<=high){
        int mid=(low+high)/2;
        if(arr[mid]==key){
            printf("Element found at position %d\n", mid + 1);
            return 0;
        }
        else if(arr[mid]<key){low=mid+1;}
        else {high=mid-1;}
    }
    printf("Element not found.\n");
    return 0;
}
```

OUTPUT:



Q4.Write a C program to implement Selection Sort.
A4. CODE:

```
#include <stdio.h>

int main() {
    int arr[100],n,i,j,min,temp;
    printf("Enter the number of elements: ");
    scanf("%d",&n);
    printf("Enter the elements:\n");
    for(i=0;i<n;i++){scanf("%d",&arr[i]);}
    for(i=0;i<n-1;i++){
        min=i;
        for(j=i+1;j<n;j++){
            if(arr[j]<arr[min]){min=j;}
        }
        temp=arr[i];
        arr[i]=arr[min];
        arr[min]=temp;
    }
    printf("Sorted array:\n");
    for(i=0;i<n;i++){
        printf("%d ",arr[i]);
    }
    return 0;
}
```

OUTPUT:

```
Enter the number of elements: 9
Enter the elements:
9 23 12 4 16 27 1 14 19
Sorted array:
1 4 9 12 14 16 19 23 27 (base) Us
```

Q5. Merge two sorted singly linked lists into one sorted linked list.
A5. CODE:

```
#include <stdio.h>

int main() {
    int arr[100], n, i, j, temp;
    printf("Enter the number of elements: ");
    scanf("%d",&n);
    printf("Enter the elements:\n");
    for(i=0;i<n;i++){scanf("%d",&arr[i]);}
    for(i=0;i<n-1;i++){
        for(j=0;j<n-i-1;j++){
            if(arr[j]>arr[j+1]){
                temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
    printf("Sorted array:\n");
    for(i=0;i<n;i++){
        printf("%d ",arr[i]);
    }
    return 0;
}
```

OUTPUT:

```
Enter the number of elements: 5
Enter the elements:
12 9 5 2 15
Sorted array:
2 5 9 12 15 (base) User@ccsf-HP-
```